

SynapseOS

An Enterprise AI Decision Intelligence Platform

Project Architecture & Engineering Report

Modular Monolith · Multi-Tenant SaaS · MLOps · Hybrid + GraphRAG · MCP · Multi-Agent Orchestration
FastAPI · PostgreSQL · Qdrant · Neo4j · LangGraph · Docker · GCP · GitHub Actions

Prepared by

Jeswin K Reji

AI/ML Engineer | Data Scientist
jeswinkr7@gmail.com | [LinkedIn](#) | [GitHub](#)

26 July 2026

Table of Contents

Table of Contents	2
Executive Summary	5
1. About SynapseOS	6
1.1 Vision	6
1.2 Motivation	6
1.3 Objectives	6
1.4 Scope	6
1.5 Design Philosophy	7
2. Problem Statement & Industry Context	7
2.1 Challenges in Enterprise Decision-Making	7
2.2 Limitations of Traditional Business Intelligence Platforms	7
2.3 The Shift Toward AI-Native Platforms	7
2.4 Problem Statement	8
3. Product Overview	8
3.1 Core Platform Capabilities	8
3.2 Target Users	8
3.3 Typical User Workflow	9
3.4 Product Differentiators	9
3.5 Client Experience	9
4. System Architecture	9
4.1 Architectural Style: Modular Monolith	9
4.2 High-Level Platform Architecture	10
4.3 Technology Stack	10
4.4 Why Not Microservices From Day One?	11
5. Multi-Tenancy, Authentication & Security	11
5.1 Tenant Model	11
5.2 Authentication Flow	12
5.3 Registration & Tenant Provisioning	12
5.4 Role-Based Access Control	12
5.5 Data Protection Principles	13
6. Data Architecture	13
6.1 Polyglot Persistence Strategy	13
6.2 Data Lifecycle	13
6.3 Multi-Tenant Data Isolation	14

6.4 Benefits of This Approach	14
7. Module-by-Module Architecture.....	14
7.1 Identity & Access Management (Auth, Tenant, Users)	14
7.2 Dataset Management	15
7.3 Feature Engineering - The Platform's Shared Spine	16
7.4 Analytics.....	17
7.5 Forecasting.....	18
7.6 Prediction.....	18
7.7 Risk Analysis	19
7.8 Knowledge Intelligence (RAG)	20
7.9 Business AI Assistant (Multi-Agent Orchestration).....	21
8. MLOps & the Shared Feature Pipeline	22
8.1 Reproducibility Through Immutable Versioning	22
8.2 Shared Feature Store Behavior.....	22
8.3 Model Lifecycle.....	22
8.4 Graceful Degradation as an MLOps Principle.....	22
9. Deployment & CI/CD	23
9.1 Containerization.....	23
9.2 Cloud Deployment on Google Cloud Platform.....	23
9.3 Continuous Integration & Delivery with GitHub Actions.....	23
9.4 Scalability Characteristics.....	24
10. Testing Strategy	24
10.1 Unit Testing	24
10.2 Integration Testing.....	24
10.3 API Testing	24
10.4 Performance Testing.....	24
10.5 Business Validation	25
10.6 Knowledge Retrieval Evaluation	25
11. Cross-Cutting Engineering Principles	26
11.1 Layered Architecture, Everywhere	26
11.2 Service-Owned Transactions.....	26
11.3 Canonical Schemas Over Ad-Hoc Mapping.....	26
11.4 Registry / Strategy Pattern for Extensibility	26
11.5 Structured, Privacy-Conscious Logging	27
11.6 Fail-Fast for Data Integrity, Graceful for Availability	27
12. Challenges & Lessons Learned.....	27
12.1 Keeping a Shared Feature Pipeline Consistent	27

12.2 Heterogeneous Enterprise Datasets	27
12.3 Orchestrating a Multi-Agent Assistant	27
12.4 Balancing Monolith Simplicity with Future Scalability	28
12.5 Key Takeaways	28
13. Future Roadmap	28
13.1 Security & Identity	28
13.2 Data & MLOps	28
13.3 Intelligence & Analytics	28
13.4 Platform & Scale	29
13.5 Project Statistics	29
14. Conclusion	30

Executive Summary

Modern enterprises generate large volumes of structured and unstructured data, yet transforming that data into decisions still typically requires stitching together disconnected tools: a BI dashboard for reporting, a separate notebook or platform for forecasting and machine learning, a document search tool for institutional knowledge, and spreadsheets for risk assessment. Each tool defines business metrics slightly differently, and none of them talk to each other.

SynapseOS was built to close that gap. It is an enterprise decision-intelligence platform that unifies dataset management, descriptive analytics, time-series forecasting, predictive machine learning, business risk scoring, retrieval-augmented knowledge search, and a conversational multi-agent AI assistant into a single, coherent product: architected, implemented, and deployed end-to-end as a solo engineering effort.

The platform is built as a modular monolith: every capability is implemented as an internally decoupled module with its own router, service, repository, and domain logic, communicating through well-defined interfaces rather than shared internal state. This gives the system the operational simplicity of a single deployable application today, with a clear, low-risk path to a distributed microservice architecture as usage scales.

At the center of the platform sits a shared Feature Engineering pipeline that transforms raw, heterogeneous enterprise datasets into a canonical business schema. Every downstream intelligence module: Analytics, Forecasting, Prediction, Risk, Knowledge, and the Assistant - consumes this same standardized feature dataset, which guarantees that a metric such as "revenue" or "active customer" means exactly the same thing everywhere in the platform.

SynapseOS is fully multi-tenant, with JWT-based authentication, role-based access control, and tenant isolation enforced at the data, service, and authorization layers. Beyond classical analytics and ML - churn prediction, delivery-delay prediction, Prophet-based forecasting, and derived business risk scoring - the platform includes a hybrid RAG with knowledge Graph enrichment-based Knowledge module for enterprise document question-answering (combining dense vector search via Qdrant, BM25 keyword retrieval, and a knowledge graph), and an Assistant module built on LangGraph that orchestrates specialized business agents behind a single conversational interface, with real-time streaming of execution progress and a React-based client.

The system is containerized with Docker and deployed to a Google Cloud Platform virtual machine, with GitHub Actions driving an automated build, test, and deployment pipeline, production-oriented DevOps practice applied to a solo-built system. This report documents SynapseOS's architecture, module-level design, data and MLOps strategy, deployment approach, testing strategy, and the engineering principles and trade-offs behind it.

1. About SynapseOS

1.1 Vision

The long-term vision behind SynapseOS is for artificial intelligence to act as an active decision partner rather than a passive reporting surface. Instead of presenting a user with charts and leaving interpretation entirely to them, the platform aims to combine structured business analytics, contextual enterprise knowledge, and AI-driven reasoning so that organizations can move from reactive reporting toward proactive, explained, and actionable intelligence.

1.2 Motivation

Traditional BI tools have made organizational data more accessible, but they still require users to manually interpret dashboards, cross-reference reports, and draw their own conclusions across disconnected systems. As machine learning and large language models become standard enterprise tooling, that manual, fragmented workflow becomes the bottleneck rather than the data itself.

A recurring observation during the design of SynapseOS was that business stakeholders rarely want an isolated chart or a single model's output, they want an answer to a business question, arrived at by combining whichever analytical, predictive, or retrieval capability is relevant. That observation shaped the decision to build one integrated platform with a single conversational entry point, rather than a growing collection of independent tools.

1.3 Objectives

- Build a unified platform spanning analytics, predictive modelling, forecasting, risk assessment, knowledge retrieval, and AI-assisted decision support.
- Design a modular architecture that is easy to reason about today and straightforward to decompose into microservices later, without a rewrite.
- Centralize feature engineering so every intelligence module operates on one consistent, canonical business representation of the data.
- Implement a conversational AI assistant capable of orchestrating multiple specialized agents rather than exposing every capability as its own disconnected screen.
- Apply production-oriented engineering practice throughout: layered architecture, versioned data, structured logging, automated CI/CD.

1.4 Scope

The current implementation targets commerce and retail-style enterprise datasets (orders, customers, products, sellers, payments, reviews) as the primary supported business domain. The architecture: canonical schemas, a feature-builder registry, and a generic fallback builder is explicitly designed so additional industries can be supported by adding new builders, without modifying the core orchestration layer.

1.5 Design Philosophy

Six principles consistently shaped engineering decisions across every module in the platform:

- Separation of concerns - HTTP handling, business logic, and persistence are never mixed within a single component.
- Modularity - each module owns its domain completely and exposes a narrow, well-defined interface to the rest of the system.
- Layered architecture - Router → Service → Repository is applied uniformly, making the codebase navigable at platform scale.
- AI as a core architectural layer, not a bolt-on - the Feature Engineering pipeline, model training/inference, and the LangGraph-based assistant are first-class parts of the system design, not an afterthought wrapped around a traditional CRUD app.
- Scalability by design - stateless services, cached computation, and immutable versioned data mean the system can scale horizontally without architectural rework.
- Maintainability and clean engineering - consistent conventions (structured logging, error handling, transaction ownership) are applied identically across all eleven modules.

2. Problem Statement & Industry Context

2.1 Challenges in Enterprise Decision-Making

Enterprises today sit on more data than ever, yet turning that data into a decision is still slow. Three recurring problems motivate SynapseOS's design:

- Tool fragmentation - analytics, forecasting, ML, document search, and risk assessment typically live in separate systems, each with its own data model and its own definition of core metrics.
- Manual synthesis - even when the underlying data is correct, a human analyst is usually required to stitch together outputs from multiple tools before a decision can be made.
- Delayed insight - by the time a report is assembled from several sources, the business context it describes may already have changed.

2.2 Limitations of Traditional Business Intelligence Platforms

Conventional BI platforms excel at descriptive reporting “what happened?” but are largely disconnected from predictive and generative capabilities. Adding forecasting or ML typically means integrating a separate platform with its own data pipeline, which reintroduces the exact fragmentation problem BI tools were meant to solve. Very few platforms combine descriptive analytics, predictive modelling, and natural-language knowledge retrieval under one data model and one access-control boundary.

2.3 The Shift Toward AI-Native Platforms

The emergence of large language models and agent-based orchestration frameworks has changed what users expect from enterprise software: not just a dashboard to read, but a system they can ask questions directly,

in natural language, and receive a synthesized, explained answer. Building that experience well requires more than adding a chatbot on top of existing dashboards, it requires an underlying data and feature architecture consistent enough that an AI agent can reliably reason over it.

2.4 Problem Statement

There is a need for a single enterprise platform that ingests raw business data once, and from that single ingestion is able to (a) describe historical and current performance, (b) forecast and predict future outcomes, (c) translate those predictions into business risk, (d) answer natural-language questions over both structured data and unstructured documents, and (e) present all of the above through one consistent, conversational interface while remaining secure, multi-tenant, and operable by a small engineering team. SynapseOS is the answer this project builds toward.

3. Product Overview

3.1 Core Platform Capabilities

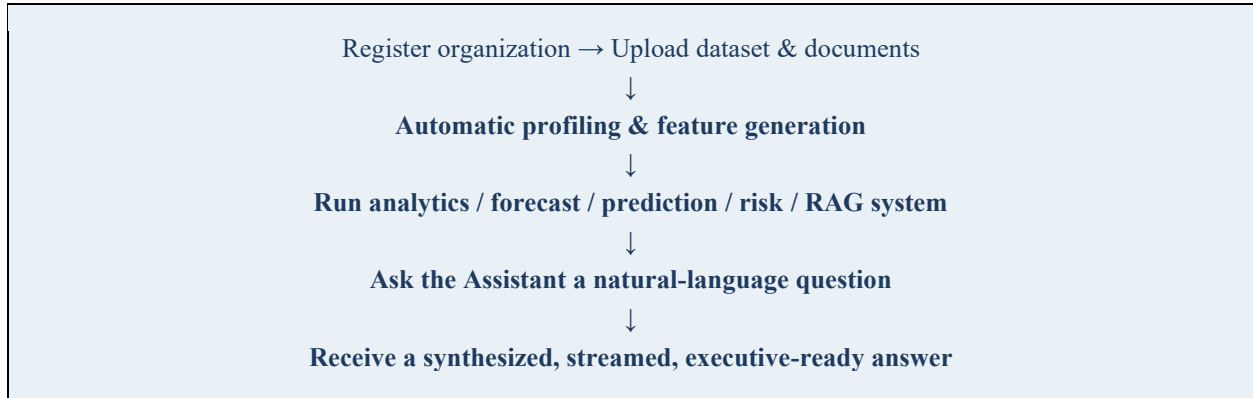
Capability	Description
Dataset Management	Versioned, multi-file dataset upload with automatic profiling and checksum tracking
Feature Engineering	Shared canonical feature pipeline consumed by every intelligence module
Analytics	Descriptive business intelligence - revenue, customers, products, sellers, operations, trends
Forecasting	Prophet-based time-series forecasting with automatic column detection and evaluation
Prediction	Supervised ML for customer churn and delivery-delay prediction, with explainability
Risk	Severity-classified business risk scoring derived from prediction outputs
Knowledge (Hybrid + GraphRAG)	Hybrid dense + keyword retrieval over enterprise documents, backed by a knowledge graph
Assistant	LangGraph-based multi-agent conversational orchestrator with streaming responses

3.2 Target Users

- Business analysts and executives who need descriptive and predictive insight without writing queries or code.

- Operations teams monitoring delivery performance, churn risk, and operational KPIs.
- Knowledge workers who need natural-language answers grounded in internal documents.
- Platform administrators managing organizational users, datasets, and access control.

3.3 Typical User Workflow



End-to-end user journey through the platform

3.4 Product Differentiators

- One canonical feature pipeline underneath every module - outputs are consistent by construction, not by convention.
- A single conversational assistant orchestrates analytics, forecasting, prediction, risk, and knowledge retrieval rather than exposing them as separate disconnected screens.
- Hybrid RAG with Knowledge Graph Enrichment (hybrid dense + BM25 retrieval with knowledge-graph context) rather than plain vector search, for materially better answer relevance.
- A modular monolith that is deliberately structured to become microservices later, avoiding both premature distributed-systems complexity and long-term architectural lock-in.

3.5 Client Experience

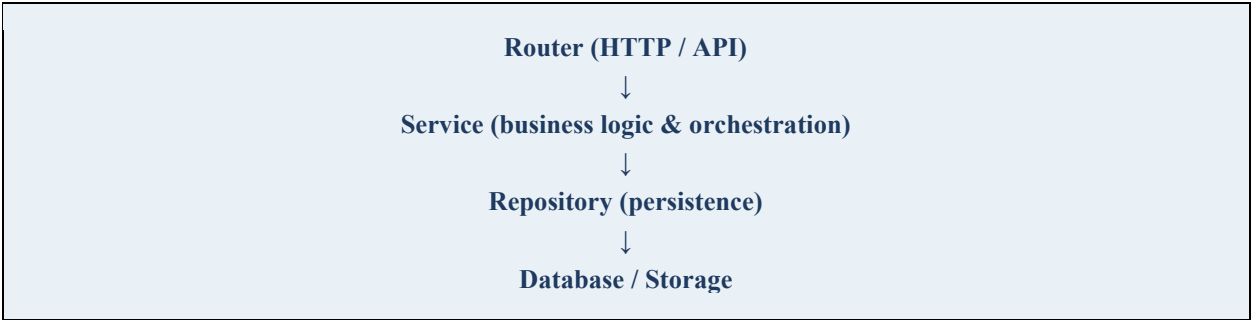
SynapseOS is accessed through a React-based web client that consumes the platform's REST and streaming APIs, alongside direct API access for programmatic integrations. The client layer is intentionally kept thin: all business logic, orchestration, and AI reasoning live in the backend, so the same capabilities exposed to the web client are equally available to any other API consumer.

4. System Architecture

4.1 Architectural Style: Modular Monolith

SynapseOS follows a modular monolith architecture. Rather than physically separating each capability into its own deployed service from day one, all modules are built and deployed as a single application, but internally structured as if they were independent services.

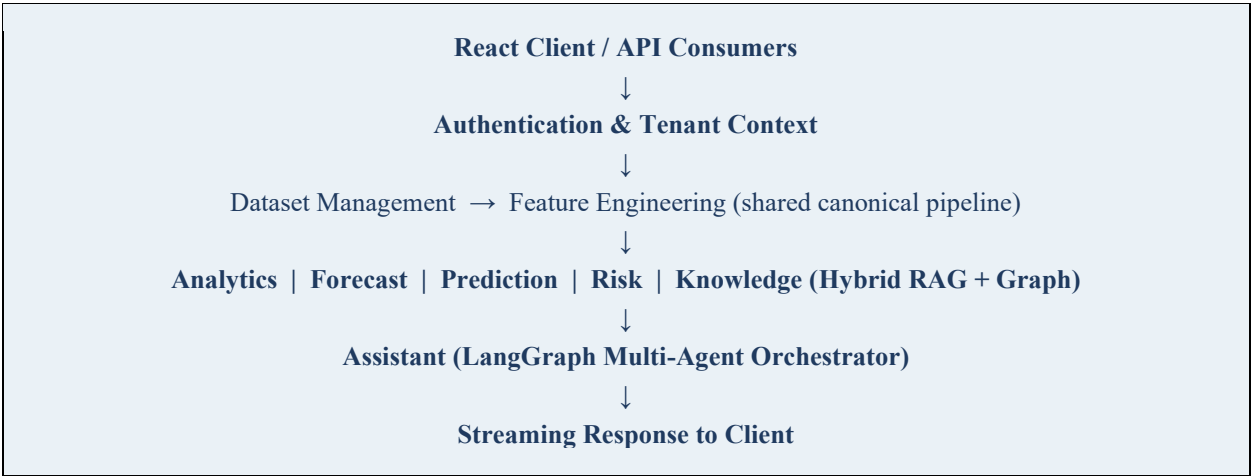
Every module in the platform follows the same internal layering convention:



Standard per-module layering used consistently across the platform

This uniform layering is what makes the monolith microservice-ready: a module's Router becomes an API gateway route, its Service becomes the new service's core logic, and its Repository becomes that service's own data-access layer. Because modules avoid reaching into each other's internal state, communicating instead through service-level calls, extracting any module into its own deployable unit is a matter of introducing a network boundary, not rewriting business logic.

4.2 High-Level Platform Architecture



SynapseOS platform-level data and control flow

Every intelligence module downstream of Feature Engineering is independent of the others at the compute level - Analytics, Forecast, Prediction, and Risk can all run concurrently against the same feature dataset but they are unified at the experience level by the Assistant, which is the single entry point most end users interact with through the React client.

4.3 Technology Stack

Layer	Technology
Client	React (web application) consuming REST and streaming APIs
API Framework	Python, layered routers / services / repositories
Primary Database	PostgreSQL
Object Storage	MinIO (S3-compatible)
Vector Database	Qdrant
Knowledge Graph	Neo4j graph database for document/entity/chunk relationships
Forecasting Engine	Facebook Prophet
ML Models	Supervised classifiers for churn and delivery-delay prediction, with explainability
Knowledge Retrieval	Hybrid Dense (Qdrant) + BM25 retrieval, Reciprocal Rank Fusion, Cross-Encoder reranking
AI Orchestration	LangGraph multi-agent workflows, Model Context Protocol (MCP)
Auth	JWT access & refresh tokens, RBAC
Containerization	Docker
Deployment	Google Cloud Platform - Compute Engine VM
CI/CD	GitHub Actions

4.4 Why Not Microservices From Day One?

As a solo-built system, a modular monolith was the pragmatic choice: it avoids the operational overhead of managing many independently deployed services - service discovery, distributed tracing, inter-service network failure modes, while a single developer iterates on the product. The layered, dependency-explicit design of each module means that decision is reversible: SynapseOS is deliberately built so that "modular monolith now, microservices later" is an engineering plan, not a hope.

5. Multi-Tenancy, Authentication & Security

5.1 Tenant Model

Every organization on the platform is represented by a Tenant, the top-level ownership boundary for all business resources - users, datasets, documents, predictions, analytics runs, and assistant conversations. A

tenant is provisioned automatically during organization registration, so tenant isolation exists from the very first authenticated request.

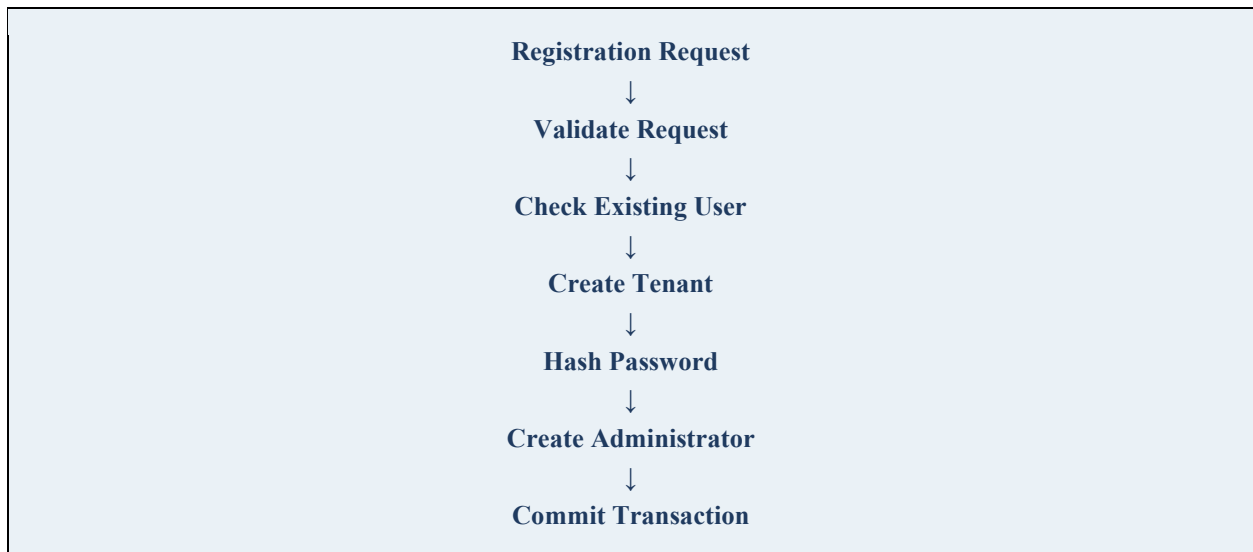
5.2 Authentication Flow



Authentication sequence

SynapseOS issues two token types: a short-lived Access Token that authorizes API requests, and a database-backed Refresh Token that mints new access tokens after expiry. Persisting refresh tokens rather than relying purely on stateless JWTs enables secure logout, explicit revocation, and a foundation for future device management.

5.3 Registration & Tenant Provisioning



Organization onboarding creates a tenant and its first administrator atomically

5.4 Role-Based Access Control

Three roles govern access across the platform: Administrator, Executive, and Analyst. Protected endpoints validate both authentication and the caller's role before any business logic executes, keeping authorization a cross-cutting concern rather than logic scattered through individual services.

Operation	Required Role
Create User	Administrator
List Users	Administrator, Analyst
Register Organization	Public (creates first Administrator)
Dataset & Intelligence Operations	Authenticated, tenant-scoped

5.5 Data Protection Principles

- Passwords are hashed before persistence and never stored or logged in plain text.
- JWT access tokens and refresh tokens are excluded from all structured logs.
- Tenant isolation is enforced at the service layer for every dataset, document, prediction, and analytics operation - cross-tenant access is structurally prevented, not merely filtered at the UI.
- Structured logging follows a consistent '<Action> | key=value key=value' convention across every module, capturing business events while excluding sensitive payloads.

6. Data Architecture

6.1 Polyglot Persistence Strategy

Rather than forcing every kind of data into a single database, SynapseOS deliberately uses different storage engines for different data shapes, each chosen for what it is good at:

Store	Data Held	Why This Store
PostgreSQL	Tenants, users, datasets metadata, predictions, forecasts, conversations	Strong consistency, relational integrity, transactional guarantees
MinIO (object storage)	Raw uploaded dataset files, forecast/model artifacts	Cheap, scalable binary storage; keeps large blobs out of the relational database
Qdrant (vector database)	Document chunk embeddings for semantic retrieval	Purpose-built for fast approximate nearest-neighbor vector search at scale
Neo4j (Graph)	Entities, chunks, and their relationships (HAS_CHUNK, MENTIONED_IN, NEXT_CHUNK)	Captures structural and semantic relationships that flat vector search cannot express

6.2 Data Lifecycle

Data moves through the platform in a consistent shape regardless of which module ultimately consumes it: raw upload → validation → storage (PostgreSQL metadata + MinIO blob) → profiling → canonical feature

generation → module-specific consumption (analytics, forecast, prediction, risk) or, for documents, chunking → embedding → indexing into Qdrant and the neo4j knowledge graph → retrieval.

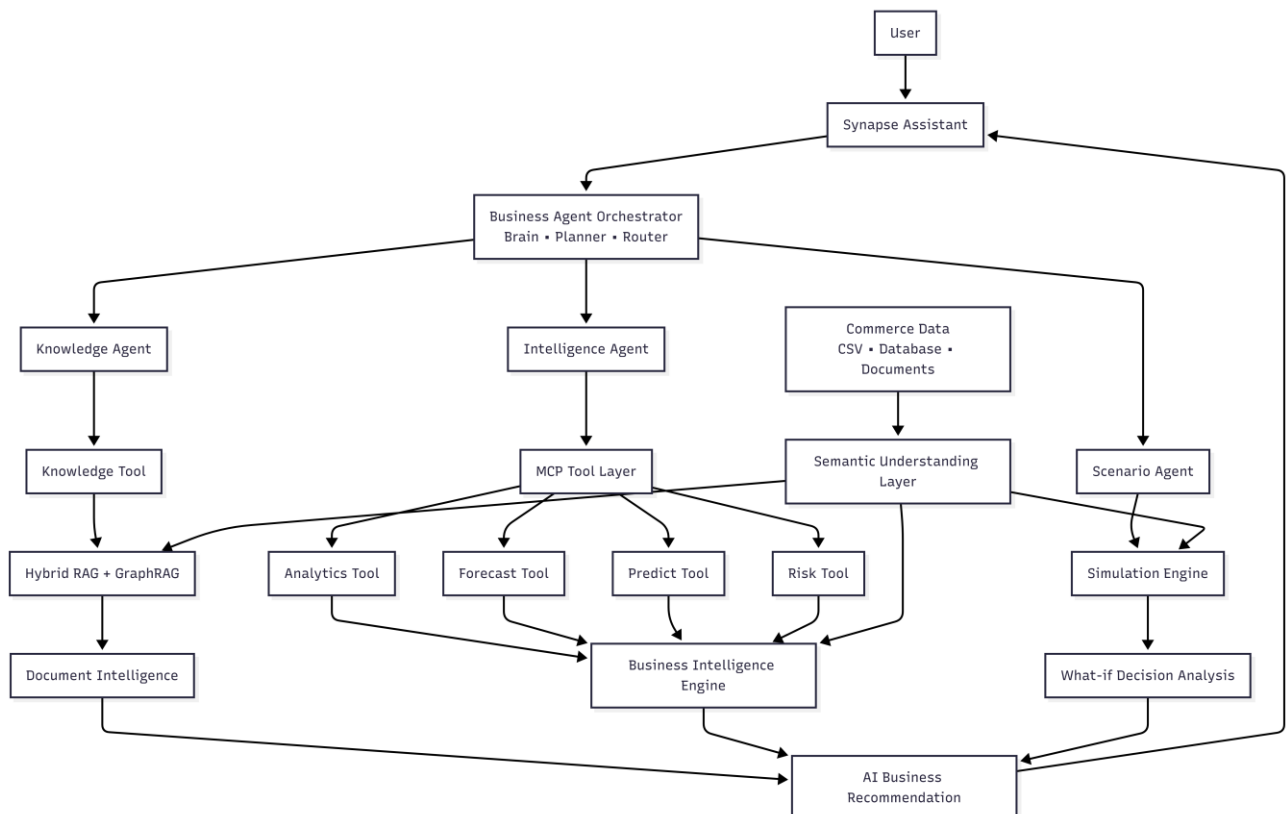
6.3 Multi-Tenant Data Isolation

Every row and every object in every store carries a tenant identifier, and every repository-level query is tenant-scoped by construction rather than by convention. This means cross-tenant access is prevented at the data-access layer itself, not merely hidden by the UI or filtered late in a service call.

6.4 Benefits of This Approach

- Each data shape lives in the store best suited to it, instead of forcing vectors, blobs, and relational data into one general-purpose database.
- Large binary content never touches PostgreSQL, keeping the relational database fast and easy to back up.
- Vector search performance scales independently of transactional workload, since Qdrant is a dedicated engine rather than a bolt-on extension.
- The knowledge graph adds relationship-aware retrieval that a pure vector or pure relational store cannot provide on its own.

7. Module-by-Module Architecture



7.1 Identity & Access Management (Auth, Tenant, Users)

The IAM layer is the platform's entry point and is composed of three tightly related modules: Tenant (organization onboarding), Authentication (login, tokens, sessions), and User Management (tenant-scoped user CRUD under RBAC).

Tenant Module

Provides a single endpoint, POST /tenants, guarded by a uniqueness rule on company name, and acts as the ownership root for every other resource in the system.

Authentication Module

Exposes registration, login, token refresh, and logout. Business logic (password verification, token issuance, transaction management) lives entirely in the service layer; the router and repositories stay deliberately thin.

User Management Module

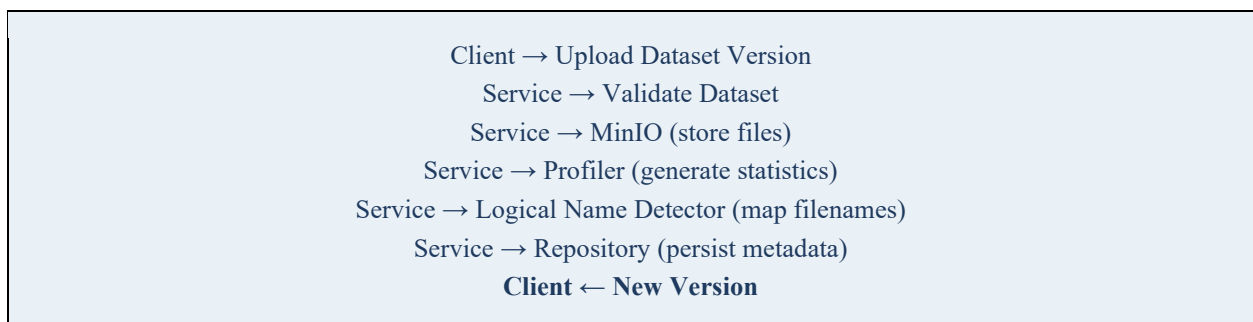
Scopes every user operation to the authenticated tenant, enforces unique emails per platform, and gates creation/listing behind RBAC, keeping authorization checks outside the service layer so business logic never has to re-implement access rules.

Design Highlights

- Service-owned transactions: repositories stay focused purely on persistence.
- Stateless JWT authentication supports horizontal scaling without server-side session storage.
- Tenant-first provisioning guarantees isolation from the very first authenticated call, not as a retrofit.

7.2 Dataset Management

The Dataset module manages the full lifecycle of enterprise datasets: creation, immutable versioning, multi-file upload, automatic profiling, checksum generation, and logical filename detection - the substrate every downstream intelligence module reads from.



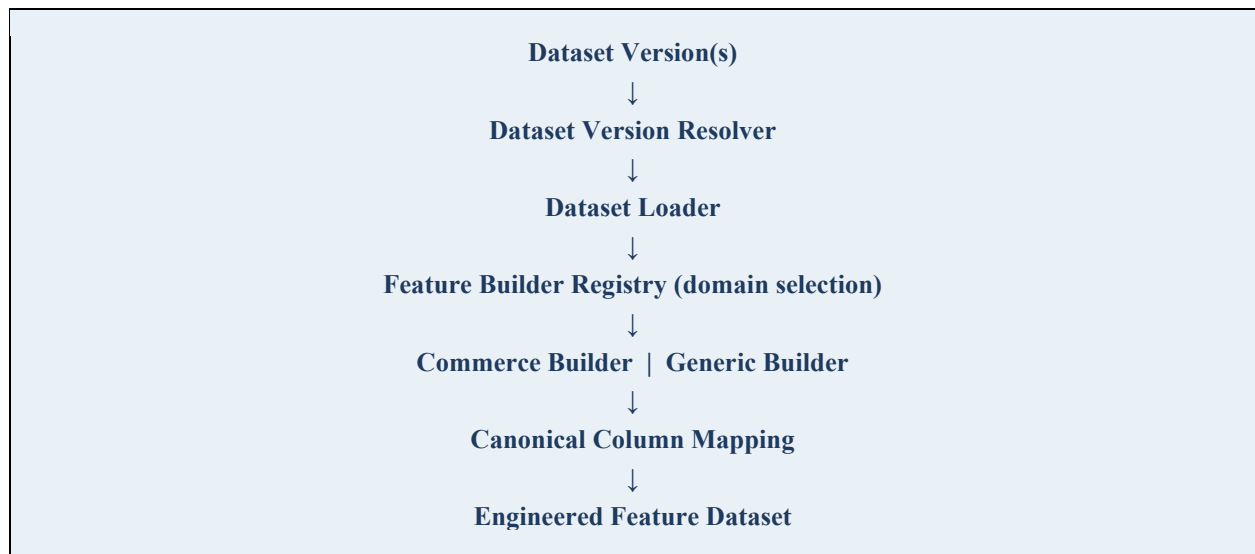
Dataset version upload pipeline

Key Design Choices

- Every upload creates a new immutable version rather than overwriting data, enabling reproducibility, rollback, and historical comparison.
- Large files live in MinIO object storage; PostgreSQL stores only metadata (path, checksum, size, MIME type), keeping the database lean.
- A Logical Name Detector maps arbitrarily-named uploaded files (e.g. sellers_final.csv) to canonical business names (sellers), so ML pipelines never depend on user-chosen filenames.
- Automatic profiling runs immediately on upload, eliminating a manual profiling step before analytics or ML can begin.

7.3 Feature Engineering - The Platform's Shared Spine

Feature Engineering is arguably the most architecturally important module in SynapseOS: it is the single shared transformation layer that every other intelligence module - Analytics, Forecast, Prediction, Risk, and the Assistant depends on. Rather than each module implementing its own data-cleaning and transformation logic, all business feature generation is centralized here.



Feature generation pipeline

Canonical Schema & Domain-Driven Builders

A registry pattern selects the correct feature builder based on the dataset's declared business domain (currently Ecommerce and Retail), with unsupported domains automatically falling back to a Generic Feature Builder, so feature generation never hard-fails for a new industry, it simply degrades gracefully until a specialized builder is written.

Multi-File & Multi-Version Aggregation

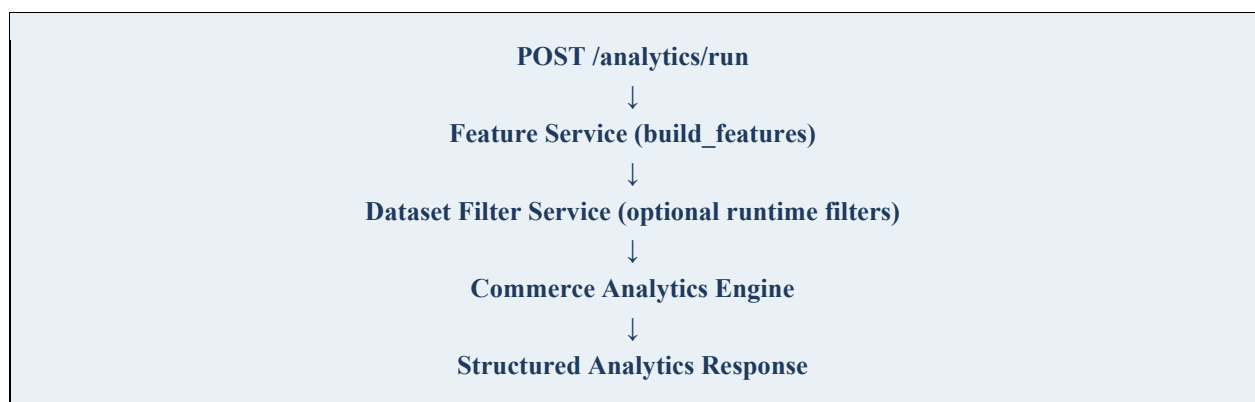
Commerce datasets typically arrive as multiple related tables - orders, customers, products, sellers, payments, reviews. The Commerce Feature Builder merges these into one unified feature table automatically, and the Feature Service can further combine multiple dataset versions into a single feature set for cross-version and comparative workflows.

Why This Matters

- Every downstream module consumes identical canonical features, no drift between what Analytics calls 'revenue' and what Prediction calls 'revenue'.
- New business domains can be added by writing a new builder, without touching the orchestration layer or any downstream module.
- Feature results are cached, so repeated requests against the same dataset version avoid redundant computation.

7.4 Analytics

The Analytics module turns engineered features into descriptive business intelligence: revenue, customer behavior, product and seller performance, reviews, operations, geography, trends, and automatically generated insights.



Analytics execution pipeline

Adaptive, Modular Output

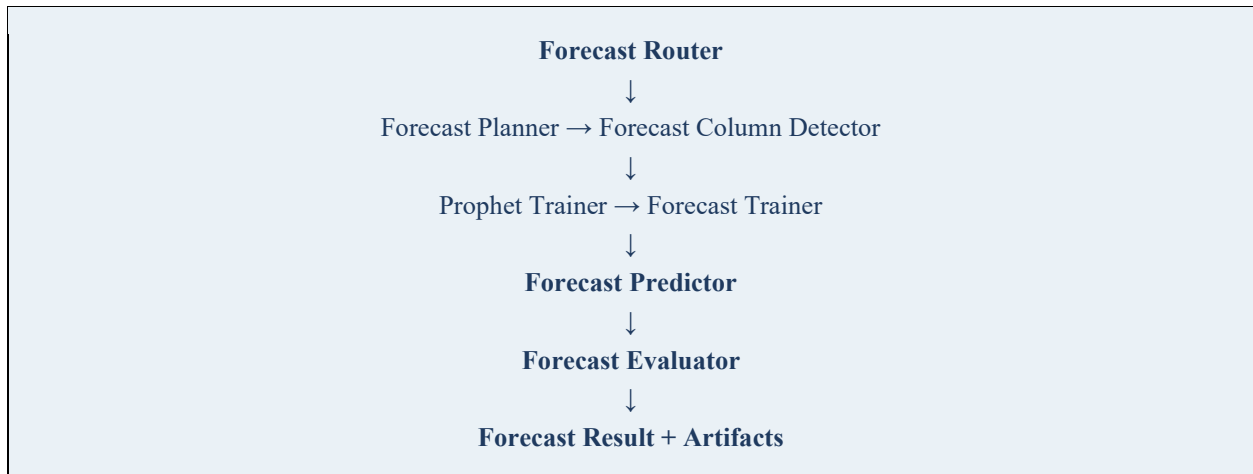
The engine dynamically detects which analytics sections it can compute based on which canonical columns are present, so a dataset missing review data simply omits the Reviews section rather than failing the entire request. Each analytics category executes independently, so partial data never blocks the rest of the report.

Design Highlights

- Deterministic, stateless engine - identical inputs always produce identical outputs, simplifying testing and caching.
- Results are cached whenever no runtime filters are applied, and filtered requests transparently bypass the cache to guarantee correctness.
- Fail-fast on feature generation errors, but cache failures degrade gracefully, analytics still returns a result, just uncached.

7.5 Forecasting

The Forecast module provides time-series forecasting over business metrics using Facebook Prophet, with dedicated components for planning, training, prediction, evaluation, and automatic detection of forecastable date/value columns.



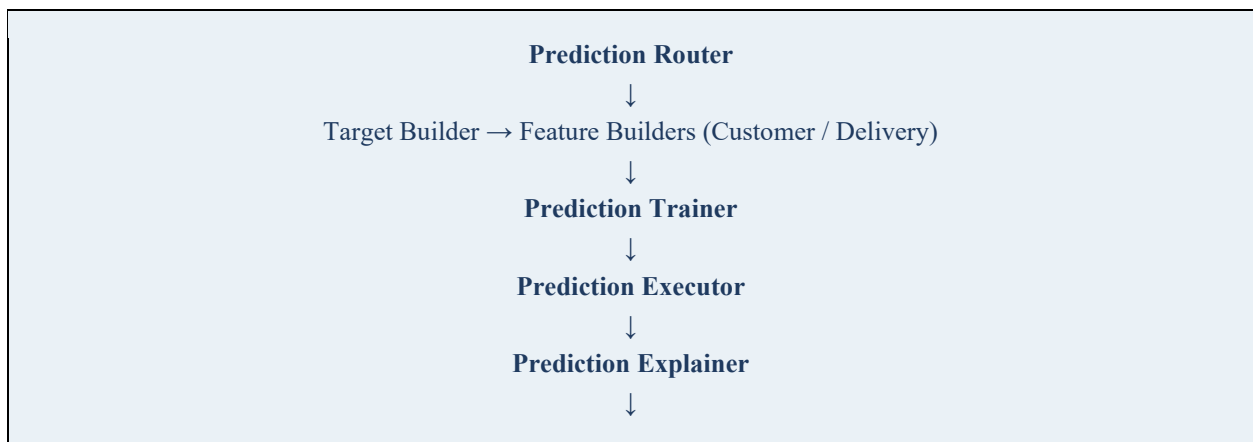
Forecast training and prediction pipeline

Capabilities

- Automatic detection of date and value columns suitable for forecasting, reducing manual configuration.
- Full training-to-prediction lifecycle with persisted forecast entities and artifacts for reproducibility.
- Evaluation metrics generated alongside forecasts to communicate model reliability, not just point predictions.

7.6 Prediction

The Prediction module runs supervised machine learning to answer two concrete business questions: which customers are at risk of churn, and which deliveries are likely to be delayed. It shares the platform's Customer and Delivery feature builders, and pairs every prediction with a human-readable explanation.



Summary + Predictions + Recommendations

Prediction execution pipeline

Supported Prediction Types

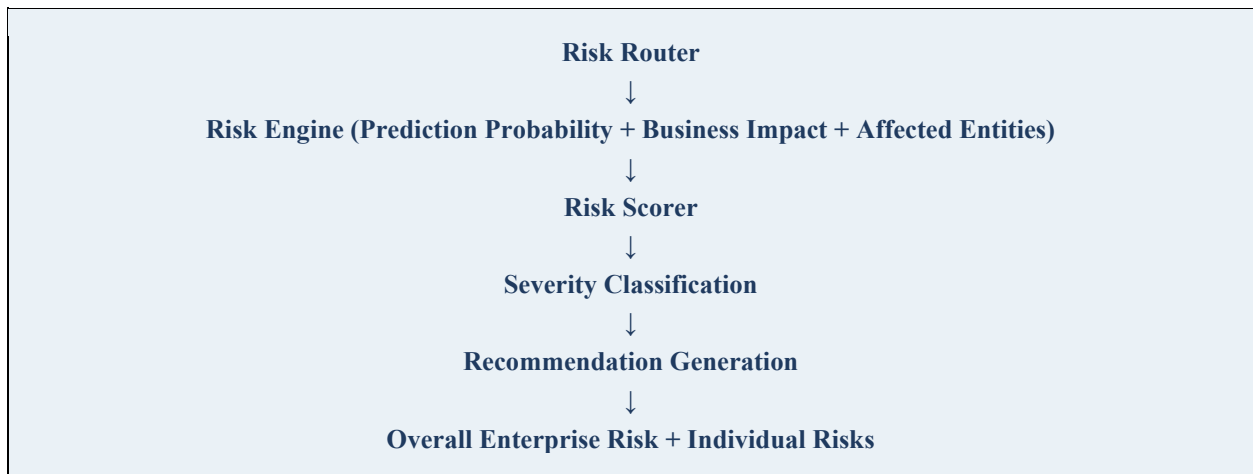
Type	Business Question Answered
Customer Churn	Which customers are likely to stop purchasing?
Delivery Delay	Which shipments are likely to arrive late?

Design Highlights

- Prediction Explainer attaches interpretable reasoning to each prediction rather than returning an opaque score.
- Model and prediction-result caching reduce repeated retraining and recomputation for identical inputs.
- Output is structured as summary, individual predictions, recommendations, and metadata - directly consumable by dashboards or the Assistant.

7.7 Risk Analysis

Risk takes the outputs of the Prediction module and translates raw probabilities into business-meaningful, severity-classified risk assessments with concrete recommendations turning 'this customer has a 0.82 churn probability' into an actionable, ranked business signal.



Risk scoring pipeline

Risk Score Composition

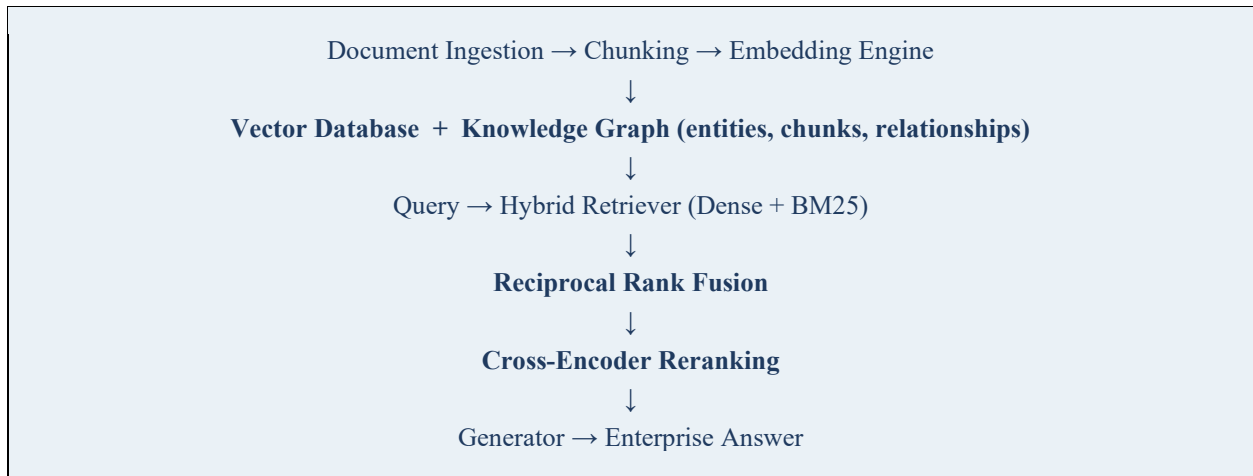
Each risk score combines three inputs: the underlying prediction probability, an estimate of business impact, and the set of affected entities, combining statistical confidence with business materiality rather than surfacing raw model probabilities alone.

Design Highlights

- Stateless execution - risk scoring is a pure function of prediction outputs and feature data.
- Produces both an aggregate 'overall enterprise risk' and a breakdown of individual risk items, supporting both executive summaries and operational drill-down.

7.8 Knowledge Intelligence (Hybrid RAG with Knowledge Graph)

The Knowledge module lets enterprise users ask natural-language questions over their own documents. It combines dense vector retrieval, classical BM25 keyword retrieval, and a knowledge graph of entities and chunks into a single hybrid retrieval-augmented generation (RAG) pipeline.



RAG retrieval and generation pipeline

Why Hybrid Retrieval

Dense (embedding) retrieval captures semantic similarity but can miss exact terminology; BM25 keyword retrieval captures exact matches but misses paraphrasing. Reciprocal Rank Fusion combines both ranked lists into a single result set, and a cross-encoder reranks the fused candidates before generation, materially improving answer relevance over either method alone.

Knowledge Graph Relationships

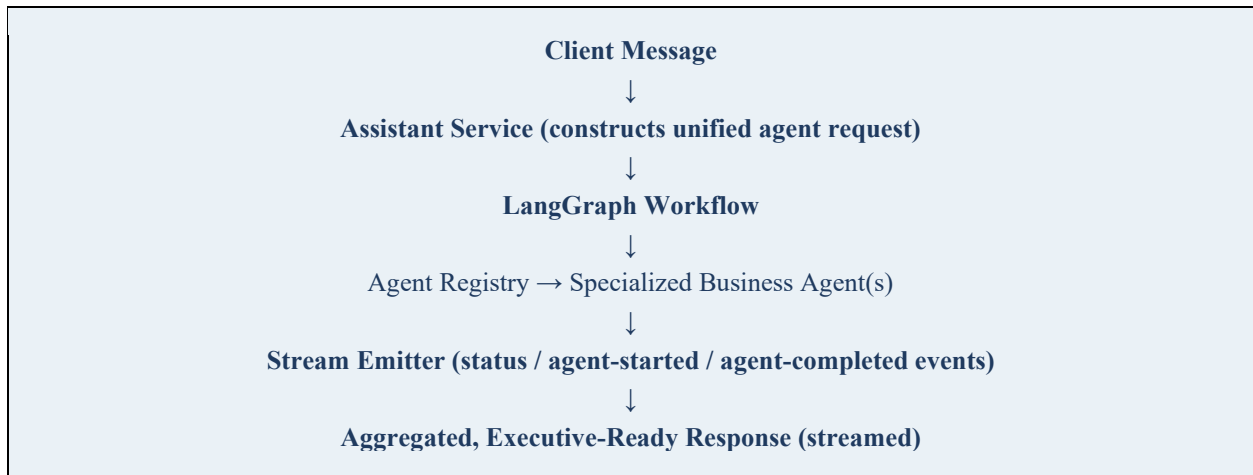
- HAS_CHUNK — links a document to its constituent chunks.
- MENTIONED_IN — links extracted entities back to the chunks that reference them.
- NEXT_CHUNK — preserves sequential document structure for context-aware retrieval.

Document Lifecycle

Documents move through explicit processing states from ingestion through chunking, embedding, and graph indexing, giving operators visibility into pipeline progress rather than a black-box 'processing' status.

7.9 Business AI Assistant (Multi-Agent Orchestration)

Rather than exposing each capability as a separate dashboard, SynapseOS unifies Analytics, Forecasting, Prediction, Risk, and Knowledge behind a single conversational Business AI Assistant. The Assistant is a modular orchestration layer built on a multi-agent architecture using LangGraph for workflow execution and the Model Context Protocol (MCP) for tool/service integration.



Assistant multi-agent execution and streaming flow

Conversation Management

Conversations and messages are persisted independently of the AI execution path, so chat history can be reused as lightweight LLM-compatible context for future agents without re-deriving it from raw application data on every turn.

Unified Request/Response Model

Regardless of which specialized agent ultimately handles a request: a Knowledge agent, an Intelligence agent and/or a Scenario agent the Assistant presents one consistent request and response schema to the client, hiding the internal multi-agent complexity behind a single interface.

Streaming Architecture

Because agent execution can involve multiple steps (feature generation, model inference, retrieval, generation), the Assistant streams status event: agent started, agent completed, final complete so users see progress in real time rather than waiting on a single opaque response.

Design Highlights

- Complete tenant isolation is preserved even as requests are routed through shared LangGraph infrastructure.
- The Assistant delegates all domain logic to specialized agents and platform modules, it deliberately contains no business logic of its own, keeping it a thin, stable orchestration layer.

8. MLOps & the Shared Feature Pipeline

MLOps in SynapseOS is not bolted on as an afterthought, it is a direct consequence of the platform's data architecture. Because every intelligence module consumes the same canonical feature dataset produced by Feature Engineering, the platform gets several MLOps properties largely for free.

8.1 Reproducibility Through Immutable Versioning

Dataset versions are immutable: every upload creates a new version rather than mutating existing data. Combined with cached, deterministic feature generation, this means any analytics result, forecast, or prediction can be traced back to an exact dataset version and reproduced later - a foundational requirement for auditability and model debugging.

8.2 Shared Feature Store Behavior

While SynapseOS does not implement a dedicated Feature Store product, the Feature Engineering module already behaves like one in practice: it resolves dataset versions, generates canonical features once, caches the result, and serves that same feature dataset to Analytics, Forecast, Prediction, Risk, and the Assistant. This is the same core value proposition a feature store provides, consistency and reuse, implemented as shared platform infrastructure rather than a separate product.

8.3 Model Lifecycle

Stage	Where It Happens
Feature generation	Feature Engineering module (shared, cached)
Training	Forecast Trainer (Prophet) / Prediction Trainer (churn, delivery delay)
Caching	Model Cache - avoids redundant retraining for identical inputs
Inference	Forecast Predictor / Prediction Executor
Explainability	Prediction Explainer attaches reasoning to every prediction
Business translation	Risk Engine converts probabilities into severity-classified business risk
Evaluation	Forecast Evaluator produces accuracy/error metrics alongside forecasts

8.4 Graceful Degradation as an MLOps Principle

Across Analytics, Feature Engineering, and Risk, the platform consistently favors graceful degradation over hard failure: an unsupported business domain falls back to a Generic Feature Builder, missing columns simply omit an analytics section rather than aborting the whole request, and cache unavailability degrades

performance rather than correctness. This design philosophy keeps the platform usable on messy, real-world enterprise data rather than requiring pristine, pre-cleaned inputs.

9. Deployment & CI/CD

9.1 Containerization

Every component of SynapseOS is containerized with Docker. Packaging the API, background workers, and supporting services as Docker images gives the platform a consistent, reproducible runtime across local development and production, and removes "works on my machine" class issues entirely.

9.2 Cloud Deployment on Google Cloud Platform

SynapseOS is deployed to a Google Cloud Platform Compute Engine virtual machine running the platform's Docker containers alongside PostgreSQL, MinIO, Neo4j and Qdrant. This VM-based approach was a deliberate choice for a solo-built platform at its current stage: it keeps infrastructure surface area small and cost-predictable while the container-first design means migrating to a managed container platform (Cloud Run or GKE) later is a deployment-target change, not an application rewrite.



CI/CD deployment flow

9.3 Continuous Integration & Delivery with GitHub Actions

An automated GitHub Actions pipeline builds, tests, and deploys the platform on every change, removing manual deployment steps and ensuring that the modular monolith is validated as a whole on every commit, the same safety net that a set of separately-deployed microservices would need per-service, obtained here with a single pipeline definition.

9.4 Scalability Characteristics

- Stateless services: Analytics, Feature Engineering, Forecast, Prediction, and Risk all avoid shared in-memory state, so any container instance can serve any request.
- Shared infrastructure (PostgreSQL, MinIO, Qdrant) is the only stateful dependency, simplifying horizontal scaling of compute.
- Caching layers (feature cache, analytics cache, model cache) reduce redundant computation under load without requiring distributed coordination.
- The modular monolith's clean module boundaries mean that if a specific module like Knowledge/RAG, which is more compute-intensive becomes a scaling bottleneck, it can be extracted into its own containerized service ahead of the rest of the system, and deployed to a managed autoscaling platform independently.

10. Testing Strategy

Testing follows the same layered philosophy as the architecture itself: each layer is validated at the level appropriate to it, so a failure is easy to localize to the router, the service, the data-access layer, or the underlying business logic.

10.1 Unit Testing

Unit tests target the service and engine layers in isolation: the Analytics Engine, Feature Builders, the Risk Scorer, and the Prediction Explainer are all designed to be deterministic and side-effect free, which makes them straightforward to unit test without standing up a database or external service.

10.2 Integration Testing

Integration tests validate the seams between layers: dataset loading through to feature generation, dataset version resolution, cache integration, and the end-to-end path from a raw uploaded dataset to a generated feature dataset consumed by a downstream module.

10.3 API Testing

API-level tests exercise the full request path: authentication, authorization, request validation, business execution, and response schema for every module's public endpoints, ensuring the contract each module exposes stays stable as internals evolve.

10.4 Performance Testing

For the Knowledge module in particular, retrieval latency and reranking throughput are tested separately from correctness, since hybrid retrieval (dense + BM25 + cross-encoder reranking) introduces a distinct performance profile from the platform's other, purely relational workloads.

10.5 Business Validation

Beyond automated tests, analytics outputs, forecasts, predictions, and risk scores are validated against representative commerce datasets to confirm that computed metrics (revenue totals, churn rates, delivery-delay rates, generated insights) match hand-checked expectations, a validation step that automated unit tests alone cannot guarantee, since correctness here is a business question as much as a code-correctness one.

10.6 Knowledge Retrieval Evaluation

The Knowledge module was evaluated to verify retrieval quality, ranking effectiveness, and response latency across multiple retrieval strategies. Since the quality of a Retrieval-Augmented Generation (RAG) system depends primarily on retrieving relevant context before answer generation, the evaluation focused on retrieval performance rather than language-model generation quality.

Evaluation Dataset

The evaluation corpus consisted of enterprise documents commonly encountered in business environments, including:

- Walmart Code of Conduct
- Walmart Supplier Standards
- Shopify Annual Report
- Commerce-related documentation used by the platform

The corpus contained approximately 500 indexed document chunks generated using the platform's chunking pipeline.

A benchmark consisting of 60 manually designed business questions covering policies, governance, suppliers, strategy, finance, and operational topics was used to evaluate retrieval performance.

Retrieval Pipelines

Four retrieval configurations were evaluated:

- Dense Retrieval : Dense semantic vector search using Qdrant
- Hybrid Retrieval : Dense retrieval + BM25 keyword search + Reciprocal Rank Fusion
- Hybrid + Cross-Encoder : Hybrid retrieval followed by Cross-Encoder reranking
- Hybrid + GraphRAG : Hybrid retrieval with knowledge graph relationships and Cross-Encoder reranking

Evaluation Metrics

The following retrieval metrics were used:

- Precision@5 – proportion of retrieved chunks that were relevant.
- Recall@5 – proportion of all relevant chunks successfully retrieved.
- Hit Rate – percentage of queries retrieving at least one relevant document.
- Mean Reciprocal Rank (MRR) – measures how early the first relevant document appears.
- Average Retrieval Latency – average retrieval time excluding LLM generation.

Pipeline	Precision@5	Recall@5	Hit Rate	MRR	Avg. Latency
Dense Retrieval	0.34	0.84	0.93	0.90	63ms
Hybrid Retrieval	0.33	0.84	0.92	0.90	72ms
Hybrid + Cross-Encoder	0.33	0.83	0.94	0.90	86ms
Hybrid + GraphRAG	0.33	0.81	0.90	0.89	100ms

Dense retrieval achieved the best overall performance on the current benchmark because the evaluation corpus was relatively small. Hybrid retrieval, Cross-Encoder reranking, and GraphRAG produced comparable results while adding retrieval robustness and richer contextual reasoning. These approaches are expected to provide greater benefits as the enterprise knowledge base increases in size and complexity.

11. Cross-Cutting Engineering Principles

Several design principles recur across every module in SynapseOS, giving the platform a consistent engineering identity rather than feeling like a collection of independently-built parts.

11.1 Layered Architecture, Everywhere

Router → Service → Repository is applied uniformly. HTTP concerns never leak into business logic, and business logic never hand-writes SQL — this consistency is what keeps the codebase navigable at platform scale despite covering eleven modules.

11.2 Service-Owned Transactions

Transaction boundaries are owned by the service layer in every module, with repositories staying focused purely on persistence - a small discipline that pays off in keeping multi-step operations, such as tenant-plus-administrator creation during registration, atomic and easy to reason about.

11.3 Canonical Schemas Over Ad-Hoc Mapping

Feature Engineering's canonical column mapping and Dataset Management's logical name detection both solve the same underlying problem - decoupling downstream logic from whatever names or formats a user's raw files happen to use, applied consistently rather than reinvented per module.

11.4 Registry / Strategy Pattern for Extensibility

The Feature Builder Registry and the Assistant's Agent Registry both use the same idea: select a specialized implementation based on context (business domain, task type) while keeping the orchestration layer itself

unaware of how many implementations exist. New domains and new agents can be added without touching the core pipeline.

11.5 Structured, Privacy-Conscious Logging

Every module logs business events, not internal implementation steps, in the same '<Action> | key=value key=value' convention, while explicitly excluding passwords, tokens, raw datasets, and PII from logs - a platform-wide observability standard rather than a per-module afterthought.

11.6 Fail-Fast for Data Integrity, Graceful for Availability

Feature generation and analytics fail fast on invalid or missing critical data, but degrade gracefully when a non-critical dependency like caching is unavailable, a deliberate distinction between correctness-critical failures and performance-only failures.

12. Challenges & Lessons Learned

Building an eleven-module platform solo surfaced engineering trade-offs that a larger team, or a narrower single-purpose product, would not have faced in the same way. This section reflects on the more significant ones.

12.1 Keeping a Shared Feature Pipeline Consistent

The biggest architectural risk in this platform was letting Analytics, Forecast, Prediction, and Risk each drift toward their own interpretation of the underlying data. Centralizing feature generation into a single canonical pipeline rather than letting each module do its own light transformation required more upfront design work, but it removed an entire category of bugs where two modules would silently disagree about the same business metric.

12.2 Heterogeneous Enterprise Datasets

Real-world uploaded datasets rarely arrive as a clean, single, well-named table. Supporting arbitrary filenames, multi-file relational datasets, and varying schemas meant the Logical Name Detector and canonical column mapping had to be designed for graceful degradation from the outset, rather than assuming a fixed schema and hard-failing on anything else.

12.3 Orchestrating a Multi-Agent Assistant

Wiring LangGraph-based multi-agent workflows into a request/response API required rethinking what "a response" even means: instead of a single synchronous return value, the Assistant module needed a streaming event model (agent started, agent completed, complete) so a multi-step agent execution feels responsive rather than like a long, opaque wait.

12.4 Balancing Monolith Simplicity with Future Scalability

It would have been easy to either over-engineer this as microservices from day one (slowing solo development significantly) or under-engineer it as a tangled single codebase. Enforcing the same Router/Service/Repository layering and explicit module boundaries in every module even when it was tempting to take a shortcut for a single feature was the discipline that kept the eventual microservice-extraction path realistic rather than aspirational.

12.5 Key Takeaways

- Centralizing shared logic (feature engineering) early is far cheaper than reconciling divergent implementations later.
- Designing for graceful degradation from the start is easier than retrofitting it once every module assumes clean input.
- Consistent architectural conventions, applied without exception across every module, are what make a large solo codebase maintainable.
- A streaming-first design for AI-orchestrated features materially changes perceived product quality, even when total latency is unchanged.

13. Future Roadmap

Each module's own future-enhancement backlog rolls up into a platform-level roadmap across three themes: security maturity, deeper intelligence, and operational scale.

13.1 Security & Identity

- Multi-Factor Authentication (MFA)
- OAuth2 / OpenID Connect and Single Sign-On (SSO)
- Email verification and self-service password reset
- Login rate limiting, device management, and security event monitoring

13.2 Data & MLOps

- Formal Feature Store integration with feature versioning and lineage tracking
- Async, background dataset profiling and validation rule enforcement
- Automatic schema evolution and data quality scoring
- Dataset lineage tracking end-to-end from raw upload to model output

13.3 Intelligence & Analytics

- Cross-domain and comparative analytics engines beyond commerce/retail
- Anomaly detection and statistical significance testing
- Real-time and streaming analytics dashboards

- Natural-language executive scorecards generated directly by the Assistant

13.4 Platform & Scale

- Migration from a single GCP VM to a managed, autoscaling container platform (Cloud Run or GKE) as load grows
- Selective extraction of high-load modules, such as Knowledge/RAG, into independently deployed microservices
- Distributed feature computation for very large enterprise datasets
- Expanded agent library for the Assistant, covering more specialized business reasoning tasks

13.5 Project Statistics

The current implementation of SynapseOS demonstrates the scope of the platform from both software engineering and artificial intelligence perspectives. The project was designed and developed as a production-oriented enterprise decision intelligence platform with an emphasis on modular architecture, scalability, and maintainability.

- **Development Model:** Solo-developed enterprise project
- **Architecture:** Modular Monolith with clear module separation
- **Backend Framework:** FastAPI (Python)
- **Frontend Framework:** React (Vite)
- **Database:** PostgreSQL
- **Object Storage:** MinIO
- **Vector Database:** Qdrant
- **Knowledge Graph:** Neo4j
- **Authentication:** JWT-based authentication with Role-Based Access Control (RBAC) and Multi-Tenant architecture
- **AI Assistant:** Business AI Assistant with Agentic AI orchestration
- **Knowledge Retrieval:** Hybrid GraphRAG combining semantic search, keyword search, knowledge graph enrichment, and Cross-Encoder reranking
- **Business Intelligence Modules:** Analytics, Forecasting, Prediction, Risk Analysis, Knowledge Management, AI Assistant, User Management, Tenant Management, Authentication, Dashboard, and Data Management
- **Machine Learning Models:** Statistical forecasting, predictive analytics, risk scoring, semantic embeddings, and large language models
- **Deployment:** Dockerized services deployed on Google Cloud Platform with automated CI/CD using GitHub Actions
- **Project Goal:** Deliver a production-ready Enterprise Decision Intelligence Platform capable of combining structured business analytics with enterprise knowledge retrieval through a unified conversational AI interface.

14. Conclusion

SynapseOS demonstrates that a single, solo-built platform can span the full breadth of enterprise decision intelligence with secure multi-tenant data management, descriptive analytics, forecasting, predictive ML, business risk scoring, hybrid RAG with knowledge Graph enrichment-based document Q&A, and a multi-agent conversational assistant without sacrificing architectural discipline.

Its modular monolith design gives it the deployment simplicity a single developer needs today, while its consistent layering, canonical schemas, and clean module boundaries mean the system is genuinely ready to evolve into a distributed, microservice-based architecture as usage grows. The shared Feature Engineering pipeline at its center is the platform's single most important architectural decision: it is what allows analytics, forecasts, predictions, and risk scores to all agree with each other, and what will let new business domains and new AI agents be added without re-deriving that agreement from scratch.

Containerized with Docker, deployed to Google Cloud Platform, and built on an automated GitHub Actions CI/CD pipeline, SynapseOS reflects production engineering practices - versioned data, cached and explainable ML, structured observability, and tenant-isolated security applied consistently across every module in the platform. As both a working product and a demonstration of end-to-end platform engineering, it reflects the complete lifecycle of designing, architecting, implementing, and shipping an enterprise AI system from the ground up.